



An Axiomatic Concurrency Model



```
def update(account: cown[Account], amount: int) {  
  when(account) {  A  
    if(account.balance + amount > 0) {  
      account.balance += amount  
      val id = account.id()  
      when(log) {  B  
        log.record(amount)  
      }  
    }  
  }  
}
```



PO

po



PO

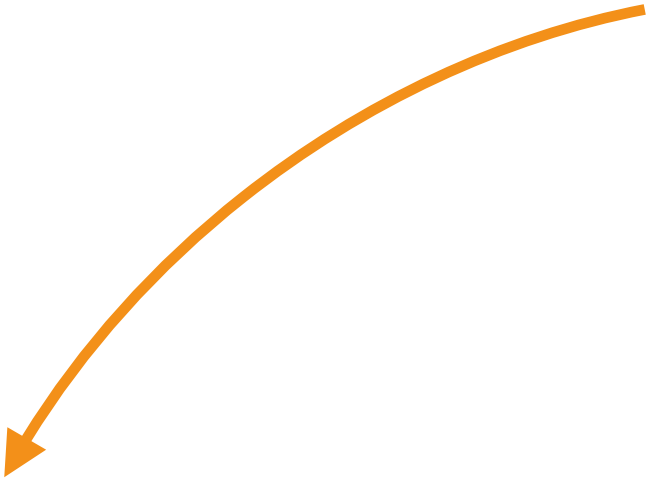
P

O

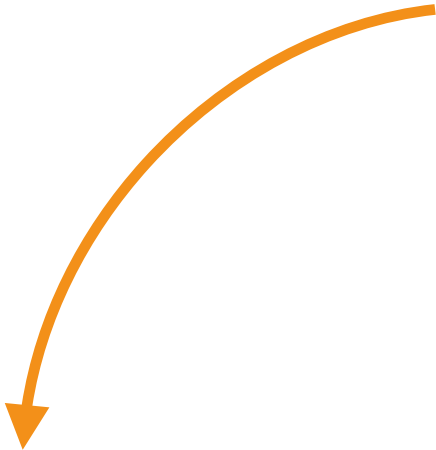


po

po



cosrc



collog

$\mathbb{E} = \text{Spawn} \cup \text{Run} \cup \text{Complete}$

$\text{comp} \subseteq \{\text{Cown} \times \text{Complete} \times \text{Run}\}$

$$\text{po} \subseteq \{\text{Run} \times \text{Spawn}\} \cup \{\text{Run} \times \text{Complete}\} \cup \\ \{\text{Spawn} \times \text{Spawn}\} \cup \{\text{Spawn} \times \text{Complete}\}$$



S



-

R  +

R



-



S



—



C



-

R  +

R.B. -



C



—

```
update(src, +100)
```

```
update(src, -100)
```

Candidate executions are tuples of (E, po, co)

An Axiomatic Concurrency Model

```

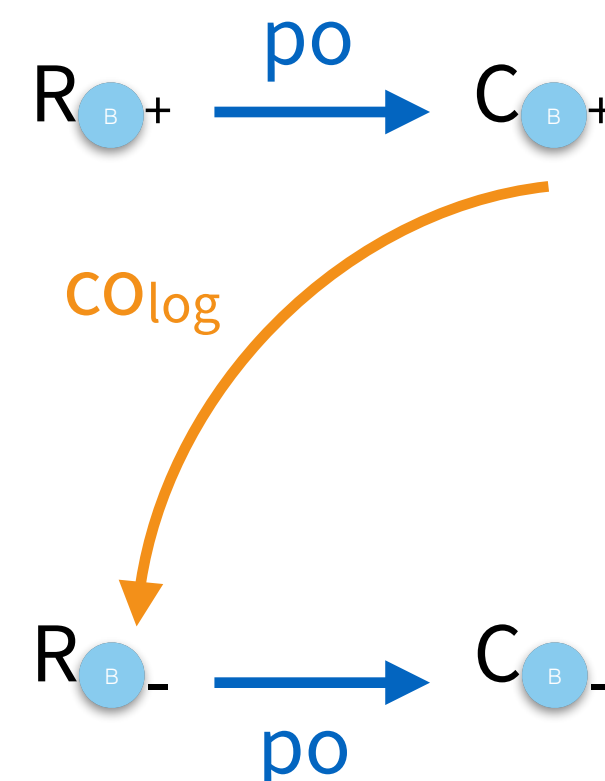
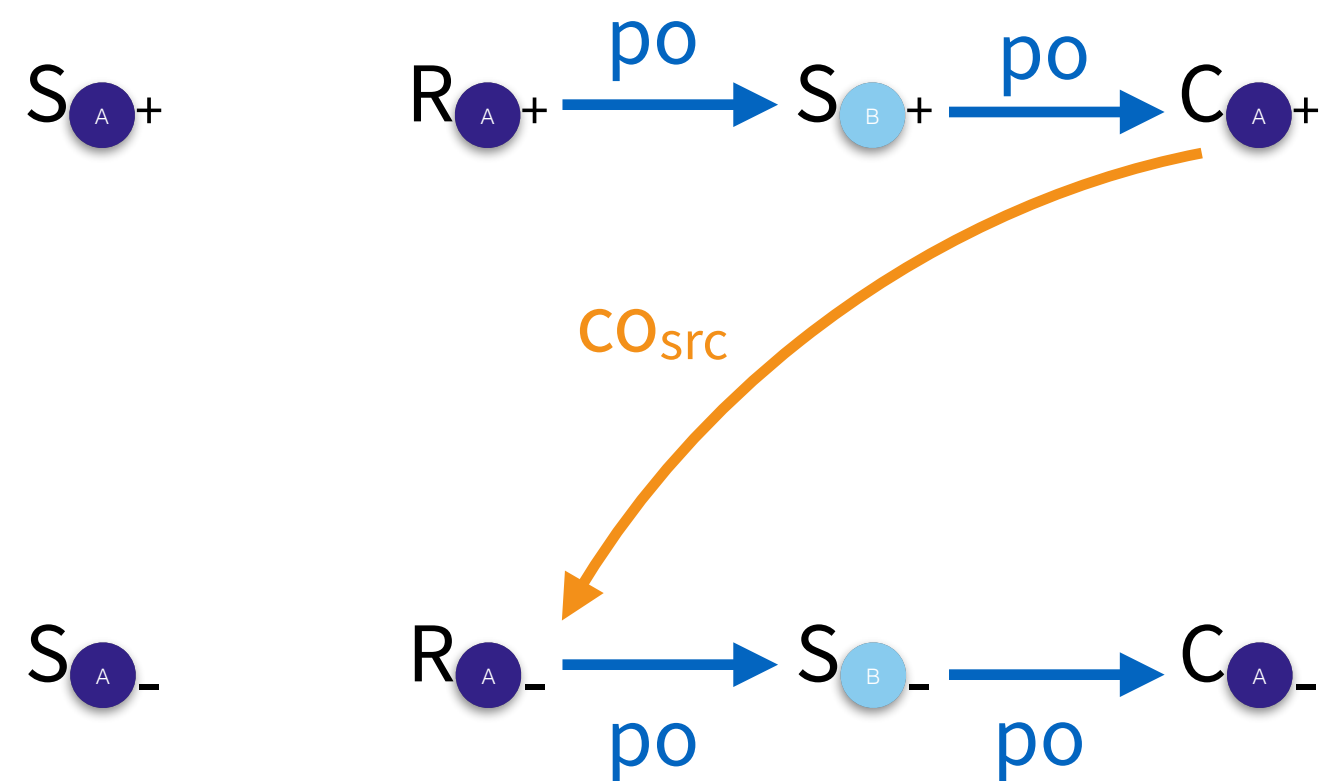
def update(account: cown[Account], amount: int) {
  when(account) { A
    if(account.balance + amount > 0) {
      account.balance += amount
      val id = account.id()
      when(log) { B
        log.record(amount)
      }
    }
  }
}

```

update(src, +100)

update(src, -100)

Candidate executions are tuples of $(\mathbb{E}, po, co)$



$\mathbb{E} = \text{Spawn} \cup \text{Run} \cup \text{Complete}$

$po \subseteq \{\text{Run} \times \text{Spawn}\} \cup \{\text{Run} \times \text{Complete}\} \cup$
 $\{\text{Spawn} \times \text{Spawn}\} \cup \{\text{Spawn} \times \text{Complete}\}$

$co \subseteq \{\text{Cown} \times \text{Complete} \times \text{Run}\}$

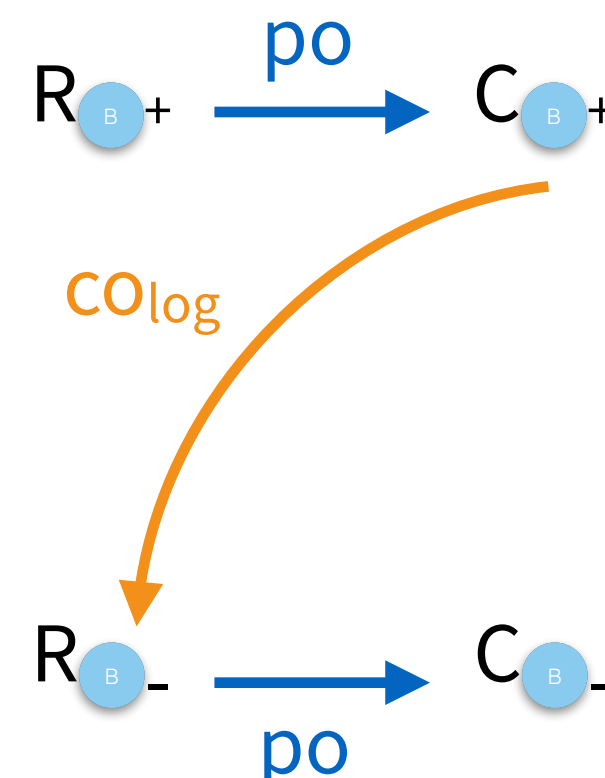
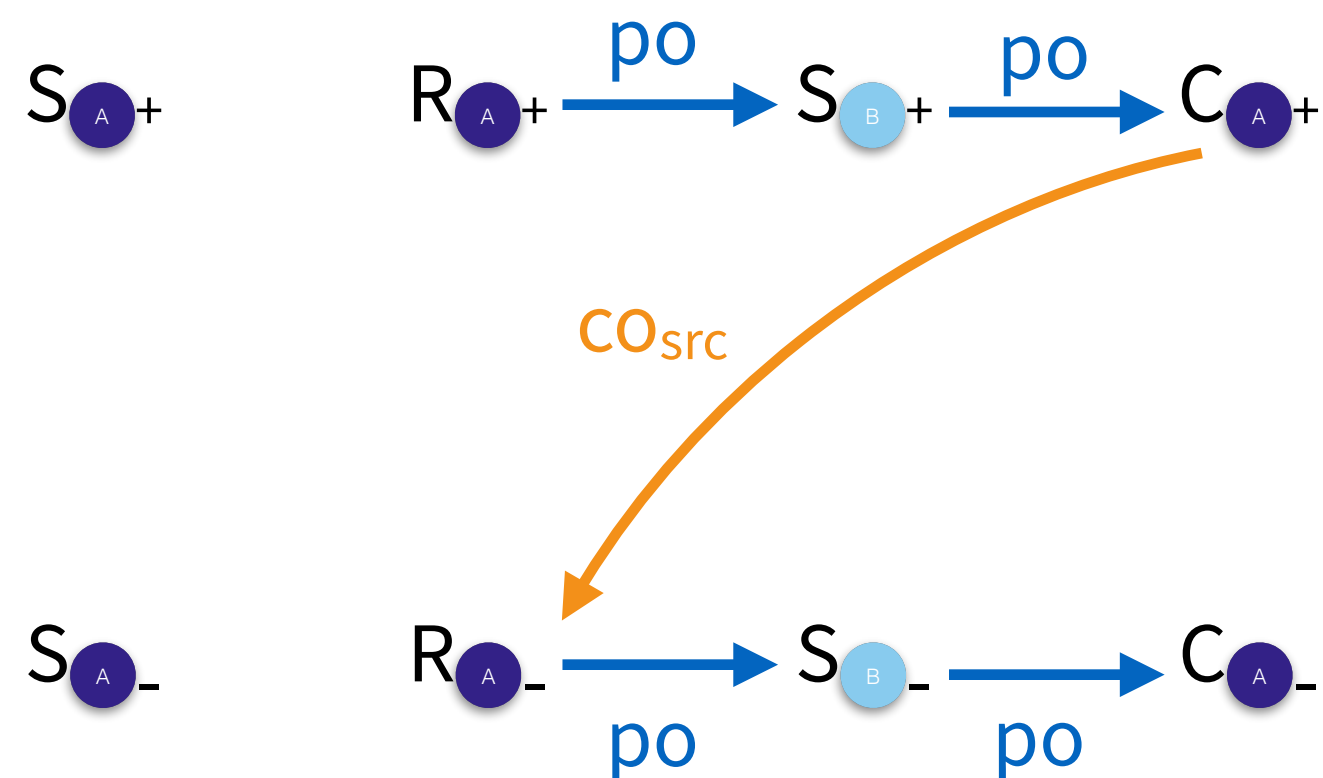
An Axiomatic Concurrency Model

```
def update(account: cown[Account], amount: int) {  
  when(account) { A  
    if(account.balance + amount > 0) {  
      account.balance += amount  
      val id = account.id()  
      when(log) { B  
        log.record(amount)  
      }  
    }  
  }  
}
```

update(src, +100)

update(src, -100)

Candidate executions are tuples of $(\mathbb{E}, po, co)$



We derive the run relation from behaviour spawn to run